

Agent Platform 的 Agents CLI：從開發到正式環境

來源：<https://codelabs.developers.google.com/agents-cli-agent-platform/agents-cli-agent-platform?hl=zh-tw>

步驟數：14

目錄

1. 總覽
2. 事前準備
3. 安裝 Agents CLI
4. 建立代理專案
5. 探索代理程式碼
6. 使用 Playground 在本機測試
7. 從指令列執行
8. 評估代理
9. 新增 Agent Runtime 部署作業
10. 部署至 Agent Runtime
11. 測試及監控已部署的代理
12. 選用：發布至 Gemini Enterprise
13. 清除所用資源
14. 恭喜！

步驟 1 · 約 2 分鐘

1. 總覽

課程內容

在本程式碼實驗室中，您將使用 Agents CLI 和 Agent Development Kit (ADK)，建構、測試及部署可供正式環境使用的 AI 代理。從安裝工具到在 Google Cloud 的 Agent Runtime 上執行即時代理程式，不到一小時即可完成。

建構項目

可執行下列動作的客服 AI 代理：

- 使用自然語言回答問題
- 呼叫自訂工具 (天氣和時間查詢)
- 在本機接受測驗，並取得即時回饋
- 自動評估品質
- 在 Google Cloud 中執行正式版

使用 Agents CLI 的兩種方式

Agents CLI 支援兩種工作流程：

使用程式碼代理 (適合初學者)

讓 AI 協助你！將技能安裝到 [Gemini CLI](#)、[Antigravity](#)、[Claude Code](#)、[Cursor](#) 或任何其他支援的程式設計代理中，這些工具會逐步協助您建構代理。

手動模式 (適合偏好直接控制的開發人員)

從終端機自行執行指令。您將輸入每個指令，並確切瞭解會發生什麼情況。

在本程式碼研究室中，我們會說明這兩種做法。選擇適合自己的風格！

需求條件

必要條件：

- Python 3.11 以上版本
- [uv](#) 套件管理工具
- Node.js 18 以上版本 (適用於程式設計代理技能)
- 已啟用計費功能的 Google Cloud 專案
- 已安裝 Google Cloud SDK

僅限本機開發時選用：

- [AI Studio API 金鑰](#) (可取代 GCP，用於本機測試)

必要條件

本程式碼研究室假設您熟悉以下概念：

- 使用終端機/指令列
- Python 基本概念
- Google Cloud 控制台基本知識

不需要有 AI 代理或 ADK 相關經驗！

步驟 2 · 約 5 分鐘

2. 事前準備

設定 Google Cloud

建立或選取專案

1. 前往 [Google Cloud 控制台](#)
2. 建立新專案或選取現有專案
3. 記下「專案 ID」，後續步驟將會用到

啟用必要的 API

在終端機中執行下列指令 (或使用 Cloud 控制台)：

```
gcloud services enable aiplatform.googleapis.com \
  run.googleapis.com \
  cloudtrace.googleapis.com \
  cloudbuild.googleapis.com
```

這項做法可提供以下優勢：

- **Agent Platform** - 適用於 Gemini 模型和 Agent Runtime
- **Cloud Run** - 其他部署選項
- **Cloud Trace** - 可觀測性和監控
- **Cloud Build** - 自動建構

驗證

```
gcloud auth login
gcloud auth application-default login
```

設定環境變數

```
export GOOGLE_CLOUD_PROJECT=YOUR_PROJECT_ID
export GOOGLE_CLOUD_LOCATION=us-central1
```

將 `YOUR_PROJECT_ID` 替換為實際專案 ID。

步驟 3 · 約 3 分鐘

3. 安裝 Agents CLI

使用程式設計代理

如果您使用 Gemini CLI、Antigravity、Claude Code、Cursor 或任何其他支援的程式設計代理：

```
uvx google-agents-cli setup
```

這會安裝下列項目：

- 全域 Agents CLI 工具
- 7 項技能，可供電腦上任何支援的程式碼編寫代理程式使用，協助您建構代理程式 (技能安裝一次，所有支援的代理程式都會發現)

預期的輸出內容 (經過修剪):

RESEARCHER'S NOTE
RESEARCHER'S NOTE

Your coding agent just got an upgrade.

1. Authentication

✓ Authenticated with Google Cloud

2. CLI Installation

```

▶ uv tool install google-agents-cli
✓ Installed google-agents-cli

```

3. Skills Installation

```
► npx -y skills add https://github.com/google/agents-cli -y --all -g
```

◇ Found 7 skills

```
~/agents/skills/google-agents-cli-adk-code
```

```
~/agents/skills/google-agents-cli-deploy
```

```
~/agents/skills/google-agents-cli-eval
```

```
~/agents/skills/google-agents-cli-observability
```

```
~/agents/skills/google-agents-cli-publish
```

```
~/agents/skills/google-agents-cli-scaffold
```

```
~/agents/skills/google-agents-cli-workflow
```

技能會透過 `skills` (`npx -y skills@latest`) 安裝到 `~/.agents/skills/` , 並由電腦上所有支援的程式碼代理程式自動擷取。

手動模式

如要直接執行指令：

```
uv tool install google-agents-cli
```

驗證安裝：

```
agents-cli --version
```

預期的輸出內容：

```
agents-cli, version 0.1.2
```

步驟 4 · 約 5 分鐘

4. 建立代理專案

使用程式設計代理

請程式設計代理執行下列操作：

「Create a new ADK agent project called customer-support-agent using the prototype template」(使用原型範本建立名為 customer-support-agent 的新 ADK 代理專案)

代理程式會執行架構指令，並為您建立專案。

手動模式

使用快速模式建立與本程式碼研究室相符的專案：

```
agents-cli scaffold create customer-support-agent --prototype --yes
```

這會立即建立基本的 ADK 代理專案，其中包含本程式碼研究室所需的所有程式碼。

預期的輸出內容：

```
Agents CLI v0.1.2
> Verifying GCP credentials...
> ✓ Connected to project: YOUR_PROJECT_ID

✅ Success! Your agent project is ready.

📖 Documentation
  README:    cat customer-support-agent/README.md

💡 Tip
  Add a deployment target later with: agents-cli scaffold enhance

🚀 Get Started
  cd customer-support-agent && agents-cli install && agents-cli playground
```

為什麼要使用快速模式？ 使用 `--prototype --yes` 可確保專案與後續步驟中顯示的程式碼相符。互動模式提供更多選項 (RAG 管道、A2A 代理程式)，但可能會建立不同的程式碼。

替代做法：互動模式

如要探索其他代理程式類型，請在不使用旗標的情況下執行：

```
agents-cli scaffold create customer-support-agent
```

系統會顯示下列選項：

- **adk** - 簡易 ReAct 代理程式 (請為程式碼研究室選擇這個選項)
- **adk_a2a** - 代理對代理通訊
- **agentic_rag** - 採用 RAG 的文件問與答

注意：如果選擇「adk」以外的選項 (選項 1)，後續步驟中的代理程式碼可能與專案不符。

建立的內容為何？

```
customer-support-agent/
├── app/
│   ├── agent.py           # Your agent code (main file)
│   └── fast_api_app.py    # Development server (replaced when you add a deployment
target)
├── app_utils/             # Utilities (telemetry, etc.)
├── tests/
│   ├── unit/              # Unit tests
│   ├── integration/       # Integration tests
│   └── eval/              # Evalsets and rubric config for `adk eval`
├── Dockerfile             # Container image (removed when you switch to Agent Runtime)
├── GEMINI.md              # Coding-agent context for this project
├── pyproject.toml         # Project config & dependencies
├── README.md              # Project documentation
└── .gitignore
```

步驟 5 · 約 3 分鐘

5. 探索代理程式碼

前往專案

```
cd customer-support-agent
```

檢查代理

開啟 `app/agent.py`，代理程式定義就在這裡：

```

import datetime
from zoneinfo import ZoneInfo

from google.adk.agents import Agent
from google.adk.apps import App
from google.adk.models import Gemini
from google.genai import types

import os
import google.auth

_, project_id = google.auth.default()
os.environ["GOOGLE_CLOUD_PROJECT"] = project_id
os.environ["GOOGLE_CLOUD_LOCATION"] = "global"
os.environ["GOOGLE_GENAI_USE_VERTEXAI"] = "True"

def get_weather(query: str) -> str:
    """Simulates a web search. Use it get information on weather.

    Args:
        query: A string containing the location to get weather information for.

    Returns:
        A string with the simulated weather information for the queried location.
    """
    if "sf" in query.lower() or "san francisco" in query.lower():
        return "It's 60 degrees and foggy."
    return "It's 90 degrees and sunny."

def get_current_time(query: str) -> str:
    """Simulates getting the current time for a city.

    Args:
        city: The name of the city to get the current time for.

    Returns:
        A string with the current time information.
    """
    if "sf" in query.lower() or "san francisco" in query.lower():
        tz_identifier = "America/Los_Angeles"
    else:
        return f"Sorry, I don't have timezone information for query: {query}."

    tz = ZoneInfo(tz_identifier)
    now = datetime.datetime.now(tz)
    return f"The current time for query {query} is {now.strftime('%Y-%m-%d %H:%M:%S %Z%z')}"

root_agent = Agent(
    name="root_agent",

```

```
model=Gemini(
    model="gemini-flash-latest",
    retry_options=types.HttpRetryOptions(attempts=3),
),
instruction="You are a helpful AI assistant designed to provide accurate and useful
information.",
tools=[get_weather, get_current_time],
)

app = App(
    root_agent=root_agent,
    name="app",
)
```

核心概念

工具：代理可呼叫的 Python 函式

- `get_weather(query)` - Returns simulated weather (foggy 60°F for SF, sunny 90°F otherwise)
- `get_current_time(query)` - Returns current time (San Francisco only in this stub)

模型：`gemini-flash-latest` 是別名，會自動追蹤最新的穩定版 Gemini Flash，因此只要 Google 發布新版本，這段程式碼就能繼續運作。如要固定特定模型，請換成 `gemini-2.5-flash` (穩定版) 或 `gemini-3-flash-preview` (搶先體驗版)。

應用程式 / `root_agent`：ADK 專案會公開頂層 `App`，包裝 `root_agent`。遊樂區 `adk eval` 和 Agent Runtime 都會透過這個 `app` 物件探索代理。

Location =

"global"：部分 Gemini 搶先版模型只會從 `global` 端點提供服務，因此支架會明確設定該端點。

指令：形塑代理行為的系統提示。

步驟 6 · 約 3 分鐘

6. 使用 Playground 在本機測試

測試區提供互動式對話介面，方便您進行測試。

使用程式設計代理

請代理人：

「Start the playground for my agent」(為我的代理程式啟動遊樂場)

手動模式

先安裝依附元件

```
agents-cli install
```

這會在幕後執行 `uv sync`，將代理程式的依附元件解析並安裝到本機 `.venv`。

啟動測試區

```
agents-cli playground
```

預期的輸出內容：

```
Starting your agent playground...

Will be available at:  http://127.0.0.1:8080/dev-ui/?app=app

► uv run adk web . --host 127.0.0.1 --port 8080 --reload_agents
INFO:      Started server process
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8080
```

立即試用

1. 在瀏覽器中開啟 <http://127.0.0.1:8080/dev-ui/?app=app>
2. 試試以下提示：
 - 「舊金山的天氣如何？」
 - 「東京的天氣如何？」
 - 「舊金山現在幾點？」

觀看代理如何：

- 呼叫 `get_weather` 工具
- 呼叫 `get_current_time` 工具

- 將結果整合為自然的回覆

熱重載：遊樂場會以 `--reload_agents` 啟動，因此編輯 `app/agent.py` 並儲存變更時，系統會自動套用變更，不必重新啟動。

這個存根 `get_current_time` 只知道舊金山時區，如果要求其他時區，系統會傳回「Sorry, I don't have timezone information...」(抱歉，我沒有時區資訊...)。這是刻意設計的結果：這個小介面是用來示範工具路徑，並非實際的實作方式。

步驟 7 · 約 2 分鐘

7. 從指令列執行

測試代理，不必開啟瀏覽器。

使用程式設計代理

請代理人：

「Run the agent with the query 'What's the weather in Paris?」 (使用「巴黎的天氣如何？」查詢執行代理程式)。

手動模式

```
agents-cli run "What's the weather in San Francisco?"
```

預期的輸出內容：

```
Using project root directory: /path/to/customer-support-agent
Local server started on port 18080 (PID 30008)
  Stop with: agents-cli run --stop-server
[user]: What's the weather in San Francisco?
[root_agent]:
[tool_call: get_weather({"query": "San Francisco"})]
[tool_response: get_weather -> {"result": "It's 60 degrees and foggy."}]The weather
in San Francisco is 60 degrees and foggy.

Session: fb30f7f7-147e-4697-8aaa-706d604589fa (resume with --session-id)
```

在幕後，`run` 會啟動背景 `adk api_server` (保持溫暖約 30 分鐘)，因此後續通話速度很快。使用 `agents-cli run --stop-server` 明確停止。繼續與 `--session-id` 進行多輪對話。

`run` 指令非常適合：

- 開發中需進行快速測試
 - 指令碼和自動化
 - CI/CD 管道
-

步驟 8 · 約 8 分鐘

8. 評估代理

ADK 評估會驗證兩項獨立事項：

1. **工具軌跡**：代理是否使用正確的引數呼叫正確的工具？確定性，完全比對。
2. **回覆品質**：最終答案是否切題、實用，且以工具輸出內容為依據？由擔任評審的 LLM 評分。

您需要**兩者**。單靠評量表評分可能會通過看似正確的錯誤答案，單靠軌跡則無法判斷使用者是否獲得實用回覆。程式碼研究室評估集練習會同時評估這兩者。

這兩項設定是由下列兩個檔案決定：

- `tests/eval/evalsets/basic.evalset.json`：要重播的對話，以及**預期**的工具呼叫
- `tests/eval/eval_config.json` - 評分指標、門檻和 LLM 評估標準

編輯評估集，加入預期的工具呼叫

將 `tests/eval/evalsets/basic.evalset.json` 的內容替換為：


```

{
  "eval_set_id": "basic_eval",
  "name": "Basic Agent Evaluation",
  "description": "Validates that the agent calls the right tools AND produces a quality response.",
  "eval_cases": [
    {
      "eval_id": "weather_san_francisco",
      "conversation": [
        {
          "user_content": {"parts": [{"text": "What's the weather like in San Francisco?"}]},
          "role": "user",
          "final_response": {"parts": [{"text": "The weather in San Francisco is 60 degrees and foggy."}]},
          "role": "model",
          "intermediate_data": {
            "tool_uses": [{"name": "get_weather", "args": {"query": "San Francisco"}}],
            "tool_responses": [],
            "intermediate_responses": []
          }
        }
      ],
      "session_input": {"app_name": "app", "user_id": "eval_user", "state": {}}
    },
    {
      "eval_id": "weather_tokyo",
      "conversation": [
        {
          "user_content": {"parts": [{"text": "What's the weather in Tokyo?"}]},
          "role": "user",
          "final_response": {"parts": [{"text": "The weather in Tokyo is 90 degrees and sunny."}]},
          "role": "model",
          "intermediate_data": {
            "tool_uses": [{"name": "get_weather", "args": {"query": "Tokyo"}}],
            "tool_responses": [],
            "intermediate_responses": []
          }
        }
      ],
      "session_input": {"app_name": "app", "user_id": "eval_user", "state": {}}
    },
    {
      "eval_id": "time_san_francisco",
      "conversation": [
        {
          "user_content": {"parts": [{"text": "What time is it in San Francisco?"}]},
          "role": "user",
          "intermediate_data": {
            "tool_uses": [{"name": "get_current_time", "args": {"query": "San Francisco"}}],
            "tool_responses": [],
            "intermediate_responses": []
          }
        }
      ]
    }
  ]
}

```

```

    ],
    "session_input": {"app_name": "app", "user_id": "eval_user", "state": {}}
  }
]
}

```

`intermediate_data.tool_uses` 區塊是**預期軌跡**，也就是代理程式應呼叫的工具和引數。軌跡指標會比較這項資料與執行階段的實際情況。

編輯評量表，為這兩項指標評分

將 `tests/eval/eval_config.json` 替換為：

```

{
  "criteria": {
    "tool_trajectory_avg_score": 1.0,
    "rubric_based_final_response_quality_v1": {
      "threshold": 0.8,
      "judgeModelOptions": {"judgeModel": "gemini-flash-latest", "numSamples": 1},
      "rubrics": [
        {"rubricId": "relevance", "rubricContent": {"textProperty": "The response directly addresses the user's query."}},
        {"rubricId": "helpfulness", "rubricContent": {"textProperty": "The response is helpful and provides useful information."}},
        {"rubricId": "tool_grounded", "rubricContent": {"textProperty": "The response is grounded in the values returned by the tools (e.g. the exact temperature and weather condition) and does not invent details."}}
      ]
    }
  }
}

```

各行程式碼的作用：

- **tool_trajectory_avg_score: 1.0**：新增軌跡指標，並設定嚴格的 1.0 門檻 (每個預期的工具呼叫都必須完全相符)。系統會比較代理的實際 `function_call` 事件與 `intermediate_data.tool_uses`，並據此計算分數。
- **rubric_based_final_response_quality_v1**：執行 **LLM 做為評估者** (此處為 `gemini-flash-latest`)，根據每個評分量表以 0 到 1 的比例為最終回覆評分，然後計算平均值。平均值達到 `threshold` (0.8) 即為通過。`tool_grounded` 評分標準明確要求評審員，如果答案與工具輸出內容矛盾或有誤，就必須扣分，這是除了軌跡檢查之外，防止生成錯誤資訊的另一道防線。

使用程式設計代理

請代理人：

「Run the evaluations for my agent」(為我的代理執行評估)

手動模式

```
agents-cli eval run --all
```

`--all` 會在 `tests/eval/evalsets/` 下每 `*.evalset.json` 執行一次。如要指定其中一個，請使用 `--evalset tests/eval/evalsets/basic.evalset.json`。

預期的輸出內容 (經過修剪)：

```
  ▶ uv run adk eval ./app tests/eval/evalsets/basic.evalset.json --config_file_path
tests/eval/eval_config.json
INFO - google_llm.py - Sending out request, model: gemini-flash-latest, backend:
GoogleLLMVariant.VERTEX_AI
INFO - local_eval_set_results_manager.py - Writing eval result to file:
app/.adk/eval_history/app_basic_eval_<ts>.evalset_result.json
Using evaluation criteria: criteria={'tool_trajectory_avg_score': 1.0,
'rubric_based_final_response_quality_v1': BaseCriterion(threshold=0.8, ...)}
*****
Eval Run Summary
basic_eval:
  Tests passed: 3
  Tests failed: 0
```

每個測試案例的結果都會寫入 `app/.adk/eval_history/` 底下，每個結果檔案都會列出各項指標的分數，方便您查看哪些檢查通過或未通過。

如何解讀結果檔案

`app/.adk/eval_history/app_basic_eval_.evalset_result.json` 中的通過案例如下所示：

```
{
  "eval_id": "weather_san_francisco",
  "final_eval_status": 1,
  "overall_eval_metric_results": [
    {"metric_name": "tool_trajectory_avg_score",          "score": 1.0, "threshold": 1.0,
"eval_status": 1},
    {"metric_name": "rubric_based_final_response_quality_v1", "score": 1.0, "threshold": 0.8,
"eval_status": 1}
  ]
}
```

如果代理程式產生幻覺並呼叫錯誤的工具 (或沒有工具)，`tool_trajectory_avg_score` 就會降至 `0.0`，案件也會失敗，即使最終文字看似合理也一樣。這是評量表無法單獨提供的屬性。

其他可用指標

ADK 隨附多個內建評估工具，可新增至 `criteria`：

指標	檢查項目
<code>tool_trajectory_avg_score</code>	與預期軌跡完全相符的工具呼叫

rubric_based_final_response_quality_v1	根據評分量表評估 LLM (最終回覆)
rubric_based_tool_use_quality_v1	根據評分標準 (工具使用) 進行 LLM 評估
final_response_match_v2	LLM 評估：最終回覆是否與預期回覆在語意上相同？
response_match_score	實際與預期最終回覆之間的 ROUGE-1
safety_v1	LLM 評估：回覆的安全性
hallucinations_v1	LLM 評估：回覆內容有依據提供的脈絡資訊

步驟 9 · 約 3 分鐘

9. 新增 Agent Runtime 部署作業

Agent Runtime 是 Google Cloud 的代管無伺服器執行階段，適用於 ADK 代理。可自動處理資源調度、基礎架構和可觀測性。



使用程式設計代理

請代理人：

「Add Agent Runtime deployment to my project」(在專案中新增 Agent Runtime 部署作業)



手動模式

```
agents-cli scaffold enhance --deployment-target agent_runtime --yes
```

預期的輸出內容：

```
Agents CLI v0.1.2
Resolved project root to: /home/user/customer-support-agent

Generating templates for comparison...
- Original template...
- Enhanced template...

Comparing files...

Will auto-update (unchanged by you):
✓ README.md
✓ app/app_utils/telemetry.py

Skipping (your code):
- app/agent.py

Files to add:
+ app/agent_runtime_app.py
+ deployment_metadata.json
+ tests/integration/test_agent_runtime_app.py

Files to remove:
- Dockerfile
- app/fast_api_app.py
- tests/integration/test_server_e2e.py

Dependency changes:
+ Add: google-cloud-aiplatform[evaluation,agent-engines]>=1.130.0
+ Add: protobuf>=6.31.1,<7.0.0

📦 Creating backup before modification...
Backup created: /home/user/.agents-cli/backups/customer-support-agent_20260430_001940

Updated: 2 files
Added: 3 files
Removed: 3 files

✅ Enhance complete!
```

有何不同之處呢？

新增時間：

- `app/agent_runtime_app.py` - Agent Runtime 包裝函式
- `deployment_metadata.json` - 部署作業追蹤
- Agent Runtime 專屬測試

已移除：

- `Dockerfile` - 不需要 (系統會管理 Agent Runtime)
- `fast_api_app.py` - 已由 Agent Runtime 應用程式取代

保留：

- `app/agent.py` - 您的代理程式碼 (未經修改！)
-

步驟 10 · 約 10 分鐘

10. 部署至 Agent Runtime

將代理程式部署至 Google Cloud 的受管理基礎架構。

更新依附元件

```
uv lock
```

使用程式設計代理

請代理人：

「Deploy my agent to Agent Runtime in project YOUR_PROJECT_ID, region us-central1」(將我的代理程式部署至專案 YOUR_PROJECT_ID 的 Agent Runtime，地區為 us-central1)

手動模式

```
agents-cli deploy --project YOUR_PROJECT_ID --region us-central1
```

預期的輸出內容：

Using project root directory: /home/user/customer-support-agent
📦 Auto-generated requirements: app/app_utils/.requirements.txt

🚢 DEPLOYING AGENT TO VERTEX AI AGENT ENGINE 🚢

📋 Deployment Parameters:

Project: YOUR_PROJECT_ID
Location: us-central1
Display Name: customer-support-agent
Min Instances: 1
Max Instances: 10
CPU: 4
Memory: 8Gi
Container Concurrency: 9

🌐 Environment Variables:

AGENT_VERSION: 0.1.0
GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY: true
GOOGLE_CLOUD_REGION: us-central1
NUM_WORKERS: 1
OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT: true

INFO:root:Introspecting app.agent_runtime_app.agent_runtime via subprocess

🚀 Creating agent: customer-support-agent (this can take 5-10 minutes)...
INFO:vertexai_genai.agentengines:Using agent framework: google-adk
Operation: projects/.../locations/us-central1/reasoningEngines/.../operations/...

[... deployment in progress for ~5-10 minutes ...]

INFO:root:Agent Runtime ID written to deployment_metadata.json

✅ Deployment successful!
Agent Runtime ID: projects/.../locations/us-central1/reasoningEngines/XXXXXXXXXXXXXXXXXXXX
Service Account: service-XXXXXXXX@gcp-sa-aiplatform-re.iam.gserviceaccount.com

📺 Open Console Playground: <https://console.cloud.google.com/vertex-ai/agents/agent->

```
engines/locations/us-central1/agent-engines/XXXXXXXXXXXXXXXXXX/playground?
project=YOUR_PROJECT_ID
```

發生什麼事了？

Agent Runtime：

- 自動封裝程式碼
- 已上傳至 Google Cloud
- 佈建的代管基礎架構 (4 個 CPU、8Gi 記憶體)
- 已設定自動調整資源配置 (1 到 10 個執行個體)
- 已啟用遙測和可觀測性

部署時間：約 5 到 10 分鐘 (一次性設定；後續重新部署速度較快)

如果中斷，請查看進度：`agents-cli deploy --status`

11. 測試及監控已部署的代理

您的代理程式現在已在 Agent Runtime 上執行！讓我們測試一下，並探索內建的監控功能。

測試代理

選項 1：控制台 Playground (最簡單)

`agents-cli deploy` 輸出內容會顯示表單的控制台 Playground 連結，格式如下：

```
https://console.cloud.google.com/vertex-ai/agents/agent-engines/locations/<REGION>/agent-engines/<RUNTIME_ID>/playground?project=<PROJECT_ID>
```

(Agent Runtime 是產品化名稱；底層 GCP 資源是 Vertex AI Agent Engine，因此網址路徑包含 `vertex-ai/agents/agent-engines`)。按一下連結即可：

1. 登入 Google Cloud 控制台
2. 透過互動式即時通訊 UI 查看已部署的代理
3. 測試查詢，例如「舊金山的天氣如何？」
4. 即時查看工具呼叫和回應

Console Playground 是驗證部署作業最簡單的方法。

選項 2：`agents-cli run -url`

您在本機使用的 `agents-cli run` 指令也會查詢已部署的代理程式：

```
RUNTIME_ID=$(jq -r .remote_agent_runtime_id deployment_metadata.json)
agents-cli run \
  --url "https://us-central1-aiplatform.googleapis.com/v1/${RUNTIME_ID}" \
  --mode adk \
  "What's the weather in San Francisco?"
```

`--mode adk` 會說 ADK SSE 通訊協定 (部署的代理程式會公開 `:streamQuery`)。 `agents-cli` 會自動附加來自有效 gcloud 憑證的 Google 存取權杖。

選項 3：Agent Engine SDK (Python)

如要透過 Python 以程式輔助方式存取，請使用 Vertex AI SDK 的 Agent Engine 模組 (`vertexai.agent_engines`)：

```

import vertexai
from vertexai import agent_engines

vertexai.init(project="YOUR_PROJECT_ID", location="us-central1")

# remote_agent_runtime_id from deployment_metadata.json (full resource name)
remote_agent = agent_engines.get(
    "projects/.../locations/us-central1/reasoningEngines/..."
)

session = remote_agent.create_session(user_id="user-1")
for event in remote_agent.stream_query(
    user_id="user-1",
    session_id=session["id"],
    message="What's the weather in San Francisco?",
):
    print(event)

```

監控代理

Agent Runtime 內建觀測功能，可透過 Google Cloud 的監控工具使用：

Cloud Trace - 要求追蹤

1. 前往「Cloud 控制台」>「Trace Explorer」
2. 選取專案
3. 依屬性篩選範圍 `service.name = customer-support-agent`
4. 深入瞭解追蹤記錄，查看模型推論、工具呼叫和端對端延遲

Cloud Logging - 應用程式記錄

1. 前往「Cloud 控制台」>「記錄檔探索工具」
2. 使用查詢：`resource.type="aiplatform.googleapis.com/ReasoningEngine"`
3. 將 `resource.labels.reasoning_engine_id=""` 新增至這個代理的範圍
4. 查看代理要求、回覆、工具執行作業和錯誤

Cloud Monitoring - 指標和資訊主頁

1. 依序前往：[Cloud 控制台](#) > [Metrics Explorer](#)
2. 依資源類型篩選 `aiplatform.googleapis.com/ReasoningEngine`
3. 實用指標：`request_count`、`request_latencies`、`instance_count`

專家級提示：Agent Runtime 會自動設定所有三種監控工具 (追蹤、記錄、監控)。不需要額外設定！

12. 選用：發布至 Gemini Enterprise

在 Gemini Enterprise 中發布代理，供貴機構使用。

須訂閱 Gemini Enterprise 才能使用

使用程式設計代理

請代理人：

「將我的代理發布至 Gemini Enterprise」

手動模式

列出專案中的 Gemini Enterprise 應用程式，取得要指定的應用程式 ID：

```
agents-cli publish gemini-enterprise --list
```

然後註冊已部署的代理程式。這項指令會從 `deployment_metadata.json` 自動讀取 Agent Runtime ID：

```
agents-cli publish gemini-enterprise \  
  --gemini-enterprise-app-id  
"projects/PROJECT_NUMBER/locations/global/collections/default_collection/engines/YOUR_APP_ID"  
\  
  --display-name "Customer Support Agent" \  
  --description "Answers weather and time questions" \  
  --tool-description "Use this tool to ask the customer support agent."
```

新增 `--interactive` (或 `-i`)，系統就會提示您輸入任何遺漏的值，不必傳遞每個旗標。

這會讓代理可供使用：

- 在 Gemini Enterprise 代理市集
- 貴機構使用者
- 集中管理及控管

詳情請參閱[發布說明文件](#)。

13. 清除所用資源

為避免產生費用，請清除您建立的資源。

刪除代理程式

從 `deployment_metadata.json` 取得 Agent Runtime 資源名稱：

```
jq -r .remote_agent_runtime_id deployment_metadata.json
```

透過 Agent Engine SDK (Python) 刪除

```
import vertexai
from vertexai import agent_engines

vertexai.init(project="YOUR_PROJECT_ID", location="us-central1")

# Full resource name from deployment_metadata.json
remote_agent = agent_engines.get(
    "projects/.../locations/us-central1/reasoningEngines/..."
)

# force=True also deletes any child sessions
remote_agent.delete(force=True)
print("✅ Deleted successfully")
```

或透過 REST API (curl)：

```
RUNTIME_ID=$(jq -r .remote_agent_runtime_id deployment_metadata.json)
curl -X DELETE \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  "https://us-central1-aiplatform.googleapis.com/v1beta1/${RUNTIME_ID}?force=true"
```

14. 恭喜！

 恭喜你達成目標！您已成功從頭開始建構、測試及部署 AI 代理程式。

您學到的內容

- 使用 Agents CLI 的兩種方式 (透過程式設計代理或手動)
- 使用 `agents-cli scaffold` 建立代理程式專案
- 建構具備自訂工具的代理
- 使用 Playground 在本機進行測試
- 執行自動評估
- 部署至 Agent Runtime (代管基礎架構)
- 使用 Cloud Trace 和 Logging 進行監控
- 正式環境最佳做法

您建構的內容

可用於正式環境的 AI 代理，具備以下功能：

- 自然語言理解 (Gemini)
- 整合自訂工具
- 品質評估
- 自動調度部署的資源
- 內建觀測功能

後續步驟

擴充代理：

- 新增更多工具 (資料庫查詢、API 呼叫)
- 搭配使用向量搜尋和檢索增強生成 (RAG)
- 新增多輪對話
- 啟用 A2A (代理對代理) 通訊

瞭解詳情：

- [Agents CLI 指南](#)
- [ADK 說明文件](#)
- [Gemini Enterprise](#)

加入社群：

- [GitHub 問題](#)
 - [Stack Overflow](#)
 - 電子郵件地址：agents-cli@google.com
 - 分享你打造的内容！
-